

The Mathematics of AksharIME: A Source-Channel Transliteration Engine

Classical foundations, empirical corrections, and the information budget
of romanized Devanagari

AksharIME Project

September 2026

Abstract

We give a complete mathematical treatment of AksharIME, a dependency-free transliteration engine for romanized Devanagari that attains 80.69% top-1 accuracy on the Aksharantar Nepali benchmark [4], exceeding the published neural state of the art, IndicXlit [5], at 80.25%. The engine is classical: a joint source-channel model [1] trained by expectation-maximization, a Kneser–Ney syllable language model [2], decoded by beam search over a syllabic lattice [9], and a corpus-frequency prior over the candidate list. We present the full derivations, four corrections to the classical treatment that proved essential (output-identity preservation in beam merging, joint backoff targets, weighted observation EM, and floating-point poisoning guards), an empirical decomposition of the *information budget* of romanized input, and the negative result that grammars trained on alignment posteriors cannot be additively fused with deterministic language models. We outline the parameter-free context-tree formulation [3] that this analysis motivates.

1 Introduction

Romanized input — typing an Indic language on a Latin keyboard — is an *information-lossy* encoding problem: the roman string $\mathcal{R}$ under-determines the target Devanagari word D (vowel length, schwa, and nasalization are not encoded). Neural sequence-to-sequence models [5] dominate published benchmarks but require frameworks, millions of parameters, and per-language retraining. This work documents a deliberately classical engine — expectation-maximization [8], Kneser–Ney smoothing [2], and weighted finite-state-style decoding [6] — that matches the neural state of the art while remaining a few megabytes of counted statistics, and states precisely where its accuracy ceiling lies: in the entropy of the roman encoding, not in the model.

Our contributions beyond the classical treatment of Li et al. [1]:

1. an output-identity-preserving beam merging rule (Proposition 1),
2. the joint backoff target for pair grammars over latent alignments (§4.2),
3. floating-point poisoning guards for forward–backward over learned tables (§4.3),
4. weighted-observation EM with an equivalence to replication (Proposition 2),
5. the corpus-frequency prior as an interpolated MAP layer (§??),
6. backward self-training via emission argmax (§??),
7. an empirical information budget of romanized Devanagari (§??).

2 Problem formulation

Let $\mathcal{R} = r_1 \cdots r_m$ be the roman input and $D = a_1 \cdots a_n$ a Devanagari word, a sequence of *aksharas* (orthographic syllables). An *alignment* σ assigns each akshara a_j a contiguous chunk s_j of $\mathcal{R}$ of length $l_j \in [1, L]$, $L = 5$, or the empty chunk; $\Sigma(\mathcal{R}, D)$ denotes the set of valid alignments. The engine computes

$$\arg \max_D P(D | \mathcal{R}) = \arg \max_D \sum_{\sigma \in \Sigma(\mathcal{R}, D)} P(\mathcal{R} | D, \sigma) P(D) = \arg \max_D P(\mathcal{R} | D) P(D),$$

the last step by independence of σ given D (total probability over the alignment space). Inference is shortest-path search over a weighted lattice in the tropical semiring $(\mathbb{R}^+ \cup \{\infty\}, \min, +)$; all probabilities are stored as $-\log$ weights.

3 The V1 engine: classical source-channel mathematics

3.1 Emission model and EM training

Following Li et al. [1], the emission model factorizes over the alignment:

$$P(\mathcal{R} | D) = \sum_{\sigma \in \Sigma(\mathcal{R}, D)} \prod_{j=1}^n P(s_j | a_j), \quad (1)$$

where each $P(s | a)$ is a multinomial over roman chunks per akshara. Since σ is unobserved, parameters are estimated by EM [8]. Define the forward and backward recurrences over the alignment lattice (1-based akshara index, $L = 5$):

$$f[0][0] = 1, \quad f[j][i] = \sum_{l=1}^{\min(L, i)} f[j-1][i-l] P(r_{i-l+1..i} | a_j), \quad (2)$$

$$b[n][m] = 1, \quad b[j][i] = \sum_{l=1}^{\min(L, m-i)} P(r_{i+1..i+l} | a_j) b[j+1][i+l], \quad (3)$$

with word likelihood $Z = f[n][m]$ and the posterior of a specific alignment edge

$$\text{post}(j, l, i) = \frac{f[j-1][i-l] P(r_{i-l+1..i} | a_j) b[j][i]}{Z} \leq 1. \quad (4)$$

The M-step with Dirichlet smoothing toward the global chunk unigram is

$$P_{\text{new}}(s | a) = \frac{c(a, s) + \alpha P_{\text{uni}}(s)}{\sum_{s'} c(a, s') + \alpha}, \quad \alpha = 0.05. \quad (5)$$

3.2 Kneser–Ney syllable language model

$P(D)$ is factorized by a trigram Kneser–Ney LM [2] over the akshara sequence, counted *deterministically* from gold segmentations (not from EM posteriors; cf. §??):

$$P_{\text{KN}}(b | a) = \frac{\max(c(a, b) - \delta, 0)}{c(a)} + \lambda(a) P_{\text{cont}}(b), \quad (6)$$

$$\lambda(a) = \frac{\delta \cdot |\{b : c(a, b) > 0\}|}{c(a)}, \quad P_{\text{cont}}(b) = \frac{|\{a : c(a, b) > 0\}| + \frac{1}{2}}{|\Sigma_{\text{bigram}}| + \frac{N}{2}}, \quad (7)$$

with absolute discount $\delta = 0.75$ [2], plus a word-initial prior $P(a | \#) = (c_{\#}(a) + \frac{1}{2}) / (W + \frac{N}{2})$. The continuation count measures the number of *distinct left contexts* of b ; it is the component that penalizes akshara sequences never observed in any context, which raw-count smoothing under-penalizes.

3.3 Decoding

The lattice DAG has nodes (roman position, LM context) and edges (chunk length, akshara) weighted $w(e) = -\log P(s \mid a)$. Exact inference is Viterbi dynamic programming in the tropical semiring $(\mathbb{R}^+ \cup \{\infty\}, \min, +)$; the engine uses beam search with width B and the k-best list feeds a re-ranking stage.

4 Corrections to the classical treatment

4.1 Output identity in beam merging

Proposition 1. *Beam states (pos, LM context) may be merged only if the spelled output strings coincide. For transliteration lattices, distinct paths sharing a context can spell distinct strings: the roman chunk **na** aligns to both aksharas na (na) and $nā$ ($nā$).*

Merging by context alone replaces the cheaper spelling with the other and the deleted candidate can never be regenerated — measured at -12 points of top-1 in the v2 decoder. The fix is a path-hash in the deduplication key (a splitmix64 avalanche of the akshara path), preserving output identity.

4.2 Joint backoff targets for pair grammars

A pair grammar $P(\pi_j \mid \pi_{j-1})$ over aligned pairs $\pi_j = (a_j, s_j)$ must back off to the *joint* unigram $P(a, s) = P(a)P(s \mid a)$, not to $P(s \mid a)$ alone: the latter assigns a rare akshara with a single observed chunk $P(s \mid a) \approx 1$, making junk paths with rare aksharas nearly free. Measured: $6.5\% \rightarrow 18.4\%$ top-1 from this correction alone.

4.3 Floating-point poisoning of forward–backward

For words the model assigns near-zero probability, Z reaches 10^{-14} or below. Although every posterior in (4) is bounded by 1 in exact arithmetic, in floating point a single degenerate word contributed $\sim 10^{14}$ units of false posterior mass, destroying the trained statistics (accuracy collapsed from 18.4% to 6.5%). The guards

$$\text{skip if } Z < 10^{-12} \text{ or } Z \notin \mathbb{R}_{\text{finite}}; \quad \text{post} := \min(\text{post}, 1)$$

restore the invariant. We state this as a required condition for any forward–backward trainer over learned tables: *the posterior bound (4) is an exact-arithmetic statement; floating-point implementations must enforce it.*

4.4 Weighted observation EM

Proposition 2. *Let a training pair carry an integer observation weight w . Then EM with all count contributions and posteriors scaled by w yields the same estimates as replicating the pair w times.*

Proof sketch. Both the corpus likelihood objective and every count in the E- and M-steps are linear in the multiplicity of a pair: replication multiplies the forward–backward statistics by w , and the M-step normalizes per pair, so the parameter update is identical. Scaling posteriors by w directly achieves the same without w -fold storage. $\square$

This lets deduplicated synthetic training files carry frequency as an explicit field; equivalence was verified empirically (identical 80.69%).

4.5 The corpus-frequency prior as interpolated MAP

The k-best list from the decoder is re-scored by

$$\text{score}'(v) = \text{score}(v) - \lambda \ln(1 + f(v)), \quad (8)$$

where $f(v)$ is the frequency of candidate v in running text. This is a log-linear mixture: the decoder contributes $\log P(v \mid \mathcal{R})$ and (8) contributes a prior interpolated between uniform and the corpus empirical distribution (exact at $\lambda = \alpha$ in the α -interpolated sense). Two quantitative findings: (i) the prior’s value is bounded by candidate depth — it re-ranks only what the decoder emitted, so the SOTA configuration required $B=256$, $k=50$ jointly (80.46%, versus 78.84% at $k=8$); (ii) the prior is informative only if the corpus frequency distribution is itself informative: mined word-pair corpora have near-uniform native-side frequencies (2.4M unique words over 2.4M tokens) and contribute nothing, while 75M tokens of running text (570k distinct words, Zipf-distributed) contribute +3.5 points.

4.6 Backward self-training via emission argmax

The inverse map $D \rightarrow \mathcal{R}$ is deterministic under the model: the canonical romanization of W is $\arg \max_s P(s \mid a)$ per akshara, and runner-up chunks are spelling variants *actually produced by users* in the corpus. Because variants are whole-akshara token substitutions, the segmentation alignment is invariant by construction; variants are weighted by joint probability and $\frac{1}{4}$ decay. This manufactures training pairs whose word distribution matches real usage (fixing the alignment bias of §?? at the data level), worth +0.23 top-1.

5 Empirical results

On the Aksharantar Nepali test split [4] (4,101 pairs; native-word subset $n=2108$), strict top-1:

System	Native top-1
IndicXlit (11M-parameter transformer) [5]	80.25%
V1 baseline (beam 64, no vocabulary prior)	75.33%
V1 + vocabulary prior ($k = 8$)	78.84%
V1 + vocabulary prior ($B=256$, $k=50$, $\lambda_{lm}=0.85$)	80.46%
+ backward self-training pairs	80.69%

Named-entity subsets improve likewise (AK-NEI 39.2% $\rightarrow$ 44.8%). Decoding latency is 10.8ms/word at beam 256 (3ms at beam 64); incremental prefix decoding (work in progress) targets the IME keystroke budget.

6 The information budget of romanized Devanagari

Define the ambiguity set $\mathcal{A}(\mathcal{R}) = \{D : P(\mathcal{R} \mid D) > \tau\}$. The oracle curves (gold present in the top- k of the decoder) are the empirical CDF of the gold rank:

$$P(\text{gold} \in \text{top-}k) : 75.3 \text{ (}k=1\text{)}, 85.2 \text{ (}2\text{)}, 88.0 \text{ (}3\text{)}, 89.7 \text{ (}5\text{)}, 92.1 \text{ (}50\text{)}.$$

The gap $75.3 \rightarrow 92.1$ is *ranking error*; the remaining 7.9 points are *coverage error*. Critically, 52% of top-1 misses differ from the gold only in vowel-length or nasal signs (*matra-only* errors): both candidates belong to $\mathcal{A}(\mathcal{R})$ with comparable priors, so *no function of $\mathcal{R}$ alone* can separate them. Each external information source shrinks the effective ambiguity set:

Evidence source	Acts on	Measured
grammar + LM (string only)	ranking within $\mathcal{A}(\mathcal{R})$	75.33%
word frequency prior	$\mathcal{A}(\mathcal{R}) \cap \text{Words}$, weighted	80.46%
word-bigram context	$P(w_i w_{i-1})$	pending (E6)
user history (on-device)	personal $P(w)$	compounding

The plateau at $\approx 80.7\%$ is thus an empirical measurement of the conditional entropy of romanized Devanagari under this grammar: it is a property of the encoding, not of the model.

7 A negative result: alignment-bias in posterior-trained grammars

Our v2 grammar conditioned transitions on the previous aligned pair $P(\pi_j | \pi_{j-1})$, trained on EM posteriors. It underperformed the classical baseline by 12 points at every fusion weight. The reason is structural: posterior mass concentrates on the *most probable alignment per corpus pair*, which is frequency-biased toward misalignments ($P(\mathbf{na} | n\bar{a}) \approx 0.97$ versus $P(\mathbf{na} | na) \approx 0.35$ makes word-initial **na**m mass onto $n\bar{a}+m\bar{a}a$ even for words whose gold aksharas are $na+m\bar{a}$). The pair table therefore encodes *alignment frequency*, not word structure, while the deterministic akshara LM encodes word structure. Additive fusion of the two injects the bias at any positive weight (measured: all $\lambda_{\text{pair}} > 0$ reduced top-1). The resolution is not a better fusion weight but a single estimator that treats context depth as latent — the context-tree weighting formulation of Willems et al. [3], which selects the context length per event by Bayesian evidence and carries a regret bound of $|T| \ln 2$ nats against the best fixed-depth model in hindsight. This is the v3 design and the subject of ongoing work.

8 Conclusion

A classical source-channel engine, with four targeted corrections to the standard treatment and a corpus-frequency MAP layer, exceeds the published neural state of the art for romanized Devanagari at 1/10th the artifact size and zero runtime dependencies. The information-budget analysis explains both the ceiling of string-only transliteration and the headroom that contextual evidence provides. All code, training data sources, and measurements are open.

References

- [1] Haizhou Li, Zhang, Y., and Su, J. A joint source-channel model for machine transliteration. In *Proc. ACL*, 2004.
- [2] R. Kneser and H. Ney. Improved backing-off for m-gram language modeling. In *Proc. ICASSP*, 1995.
- [3] F. M. J. Willems, Y. M. Shtarkov, and Tj. J. Tjalkens. The context-tree weighting method: basic properties. *IEEE Trans. Information Theory*, 41(3):653–664, 1995.
- [4] Y. Madhani et al. Aksharantar: Open Indic-language transliteration datasets and models for the next billion users. In *Findings of EMNLP*, 2023. arXiv:2205.03018.
- [5] AI4Bharat. IndicXlit: A transliteration model for Indic languages. arXiv:2205.03018; model card, <https://github.com/AI4Bharat/IndicXlit>.
- [6] L. Hellsten, B. Roark, P. Goyal, C. Allauzen, F. Beaufays, T. Ouyang, M. Riley, and D. Rybach. Transliterated mobile keyboard input via weighted finite-state transducers. In *Proc. FSMNLP*, pages 10–19, 2017.

- [7] C. Kirov, C. Johnny, A. Katanova, A. Gutkin, and B. Roark. Context-aware transliteration of romanized South Asian languages. *Computational Linguistics*, 50(2):475–534, 2024.
- [8] A. P. Dempster, N. M. Laird, and D. B. Rubin. Maximum likelihood from incomplete data via the EM algorithm. *J. Royal Statistical Society B*, 39(1):1–38, 1977.
- [9] P. Koehn. Pharaoh: a beam search decoder for phrase-based statistical machine translation models. In *Proc. AMTA*, 2004.